Activity 4.01: Car Data Classification

Activity 4.02: Random Forest Classification for Your Car Rental Company

Course: Artificial Intelligence / Machine Learning
Student: Vijay Paul Ravipati

This notebook follows the workflow from *The Applied Artificial Intelligence Workshop*. The Car Evaluation dataset contains six categorical input features and four target classes: unacceptable (`unacc`), acceptable (`acc`), good (`good`), and very good (`vgood`).

1. Import the required libraries

The activities use pandas for data handling and scikit-learn for preprocessing, model training, and evaluation.

```
import pandas as pd
from sklearn import preprocessing
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, ExtraTreesClassifier
from sklearn.metrics import classification_report, confusion_matrix
```

2. Load the Car Evaluation dataset

The dataset URL is the Packt raw CSV used in the activity. It contains 1,728 observations and six categorical features.

```
file_url = ("https://raw.githubusercontent.com/"
            "PacktWorkshops/"
            "The-Applied-Artificial-Intelligence-Workshop/"
            "master/Datasets/car.csv")

df = pd.read_csv(file_url)
df.head()
```

	buying	maintenance	doors	persons	luggage_boot	safety	class
0	vhigh	vhigh	2	2	small	low	unacc
1	vhigh	vhigh	2	2	small	med	unacc
2	vhigh	vhigh	2	2	small	high	unacc
3	vhigh	vhigh	2	2	med	low	unacc
4	vhigh	vhigh	2	2	med	med	unacc

Next steps: Generate code with `df` | New interactive sheet

3. Inspect the dataset

The six input variables are `buying`, `maint`, `doors`, `persons`, `lug_boot`, and `safety`. The `class` column is the target.

```
print("Shape:", df.shape)
print("Columns:", list(df.columns))
```

```
print("Missing values:")
print(df.isnull().sum())
```

```
Shape: (1728, 7)
Columns: ['buying', 'maintenance', 'doors', 'persons', 'luggage_boot', 'safety', 'class']
Missing values:
buying          0
maintenance     0
doors           0
persons         0
luggage_boot    0
safety          0
class           0
dtype: int64
```

4. Encode the categorical variables

Decision-tree and ensemble classifiers in scikit-learn require numerical input. Following the activity, `LabelEncoder` is used column by column to convert the categorical values into numerical labels.

```
def encode(dataframe, column):
    label_encoder = preprocessing.LabelEncoder()
    dataframe[column] = label_encoder.fit_transform(dataframe[column])
    return dataframe

for column in df.columns:
    df = encode(df, column)

df.head()
```

	buying	maintenance	doors	persons	luggage_boot	safety	class
0	3	3	0	0	2	1	2
1	3	3	0	0	2	2	2
2	3	3	0	0	2	0	2
3	3	3	0	0	1	1	2
4	3	3	0	0	1	2	2

Next steps: Generate code with `df` New interactive sheet

5. Separate features and target

The target is removed from the feature DataFrame using `pop()`, as described in the activity.

```
features = df.copy()
label = features.pop("class")

print("Feature columns:", list(features.columns))
print("Target classes:", sorted(label.unique()))
```

```
Feature columns: ['buying', 'maintenance', 'doors', 'persons', 'luggage_boot', 'safety']
Target classes: [np.int64(0), np.int64(1), np.int64(2), np.int64(3)]
```

6. Split the data into training and testing sets

The activity uses 10% of the data for testing. A fixed random state is used so that the experiment is reproducible.

```
features_train, features_test, label_train, label_test = train_test_split(
    features,
```

```
    label,
    test_size=0.1,
    random_state=168
)

print("Training rows:", len(features_train))
print("Testing rows:", len(features_test))
```

```
Training rows: 1555
Testing rows: 173
```

Activity 4.01: Car Data Classification

7. Build a decision-tree classifier

A decision tree is trained on the training data and then evaluated using the held-out test data.

```
decision_tree_classifier = DecisionTreeClassifier(random_state=168)
decision_tree_classifier.fit(features_train, label_train)

dt_predictions = decision_tree_classifier.predict(features_test)

dt_accuracy = decision_tree_classifier.score(features_test, label_test)

print("Decision Tree test accuracy:", dt_accuracy)
```

```
Decision Tree test accuracy: 0.976878612716763
```

8. Evaluate the decision tree with a classification report

The classification report provides precision, recall, F1 score, and support for each class.

```
print(classification_report(label_test, dt_predictions, zero_division=0))
```

```
              precision    recall  f1-score   support

           0       0.92      0.97      0.95        36
           1       0.89      0.89      0.89         9
           2       1.00      0.98      0.99       122
           3       1.00      1.00      1.00         6

    accuracy                           0.98       173
   macro avg       0.95      0.96      0.96       173
weighted avg       0.98      0.98      0.98       173
```

Activity 4.02: Random Forest Classification

9. Train the Random Forest classifier

The activity solution uses 100 trees, a maximum depth of 6, and `random_state=168`. Random Forest combines many decision trees to improve stability and reduce the risk of relying on one tree.

```
random_forest_classifier = RandomForestClassifier(
    n_estimators=100,
    max_depth=6,
    random_state=168
)

random_forest_classifier.fit(features_train, label_train)
rf_preds_test = random_forest_classifier.predict(features_test)
```

```
print("Random Forest test accuracy:",
      random_forest_classifier.score(features_test, label_test))
```

```
Random Forest test accuracy: 0.8959537572254336
```

10. Evaluate the Random Forest classifier

```
print("Random Forest classification report:")
print(classification_report(label_test, rf_preds_test, zero_division=0))

print("Random Forest confusion matrix:")
print(confusion_matrix(label_test, rf_preds_test))
```

```
Random Forest classification report:
              precision    recall  f1-score   support

           0       0.72      0.81      0.76        36
           1       0.00      0.00      0.00         9
           2       0.95      0.99      0.97       122
           3       1.00      0.83      0.91         6

    accuracy                           0.90       173
   macro avg       0.67      0.66      0.66       173
weighted avg       0.85      0.90      0.87       173

Random Forest confusion matrix:
[[ 29   0   7   0]
 [  9   0   0   0]
 [  1   0 121   0]
 [  1   0   0   5]]
```

11. Examine Random Forest feature importance

Feature importance shows how much each input feature contributes to the decisions made by the forest.

```
rf_varimp = random_forest_classifier.feature_importances_
rf_importance = pd.Series(rf_varimp, index=features.columns).sort_values(ascending=False)
print(rf_importance)
```

```
safety          0.352156
persons         0.345543
buying          0.128105
maintenance     0.097223
luggage_boot    0.055940
doors           0.021032
dtype: float64
```

12. Train the Extra Trees classifier

Extra Trees adds additional randomization when selecting split points. The activity uses the same number of trees, maximum depth, and random state as the Random Forest model.

```
extra_trees_classifier = ExtraTreesClassifier(
    n_estimators=100,
    max_depth=6,
    random_state=168
)

extra_trees_classifier.fit(features_train, label_train)
et_preds_test = extra_trees_classifier.predict(features_test)

print("Extra Trees test accuracy:",
      extra_trees_classifier.score(features_test, label_test))
```

```
Extra Trees test accuracy: 0.8497109826589595
```

13. Evaluate the Extra Trees classifier

```
print("Extra Trees classification report:")
print(classification_report(label_test, et_preds_test, zero_division=0))

print("Extra Trees confusion matrix:")
print(confusion_matrix(label_test, et_preds_test))
```

```
Extra Trees classification report:
              precision    recall  f1-score   support

           0       0.63      0.67      0.65        36
           1       0.00      0.00      0.00         9
           2       0.91      1.00      0.95       122
           3       1.00      0.17      0.29         6

    accuracy                           0.85       173
   macro avg       0.64      0.46      0.47       173
weighted avg       0.81      0.85      0.82       173

Extra Trees confusion matrix:
[[ 24   0  12   0]
 [  9   0   0   0]
 [  0   0 122   0]
 [  5   0   0   1]]
```

14. Examine Extra Trees feature importance

```
et_varimp = extra_trees_classifier.feature_importances_
et_importance = pd.Series(et_varimp, index=features.columns).sort_values(ascending=False)
print(et_importance)
```

```
safety          0.398494
persons         0.365881
buying          0.087897
maintenance     0.070924
luggage_boot    0.062267
doors           0.014537
dtype: float64
```

15. Compare the three models

The final comparison makes it easier to see how the single decision tree performs against the two ensemble methods.

```
comparison = pd.DataFrame({
    "Model": ["Decision Tree", "Random Forest", "Extra Trees"],
    "Test Accuracy": [
        decision_tree_classifier.score(features_test, label_test),
        random_forest_classifier.score(features_test, label_test),
        extra_trees_classifier.score(features_test, label_test)
    ]
})

comparison.sort_values("Test Accuracy", ascending=False)
```

	Model	Test Accuracy
0	Decision Tree	0.976879
1	Random Forest	0.895954
2	Extra Trees	0.849711

Conclusion

The decision tree provides a simple baseline for classifying cars into four acceptability categories. Random Forest and Extra Trees use multiple randomized trees and can provide more stable predictions than a single decision tree. The classification reports, confusion matrices, and feature-importance values provide several ways to compare the models rather than relying only on accuracy.